

The Ultimate Beginner's CI/CD Tool Guide

Understanding CI/CD is the secret weapon that separates junior developers from engineers who ship production-ready code. This guide gives you the exact tools and commands to start building automated pipelines today without getting lost in the setup.

What you'll learn:

- 1 GitHub Actions (The Modern Favorite)
- 2 GitLab CI/CD (The All-In-One Powerhouse)
- 3 Jenkins (The Classic Customizer)
- 4 Testing Locally (The Secret Shortcut)

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

GitHub Actions (The Modern Favorite)

GitHub Actions is built directly into GitHub, making it incredibly easy to start. It uses YAML files stored in your repository to automate tasks like running tests or deploying code on every push. You don't need to host any servers; GitHub runs everything in the cloud for you.

```
$name: Run Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npm test
```

Cloud & DevOps Daily

Jenkins vs GitLab CI/CD vs GitHub Actions: Which Should You Use?

Thursday, October 08, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

GitLab CI/CD (The All-In-One Powerhouse)

GitLab CI/CD is highly structured and uses a single file named `.gitlab-ci.yml`. It organizes your pipeline into clear stages like build, test, and deploy. GitLab executes these steps using lightweight agents called 'GitLab Runners' that you can run anywhere.

```
$stages:
  - test

run_tests:
  stage: test
  image: node:latest
  script:
    - npm install
    - npm test
```

Cloud & DevOps Daily

Jenkins vs GitLab CI/CD vs GitHub Actions: Which Should You Use?

Thursday, October 08, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Jenkins (The Classic Customizer)

Jenkins is the industry pioneer. Unlike GitHub Actions or GitLab, you must install and manage Jenkins on your own server. It uses a file called a 'Jenkinsfile' written in Groovy, and relies on a massive library of community plugins to connect to other tools.

```
$pipeline {  
  agent any  
  stages {  
    stage('Test') {  
      steps {  
        sh 'npm install'  
        sh 'npm test'  
      }  
    }  
  }  
}
```

Cloud & DevOps Daily

Jenkins vs GitLab CI/CD vs GitHub Actions: Which Should You Use?

Thursday, October 08, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Testing Locally (The Secret Shortcut)

Pushing your code to GitHub 50 times just to fix a spelling mistake in your pipeline is frustrating. By using the 'act' tool, you can run your GitHub Actions workflows locally on your own terminal to catch mistakes in seconds.

```
$act -j test
```

Cloud & DevOps Daily

Jenkins vs GitLab CI/CD vs GitHub Actions: Which Should You Use?

Thursday, October 08, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Keep your secrets and API keys out of your code by using built-in environment variables.
- + Always use specific version tags for your actions and Docker images instead of using 'latest'.
- + Run your automated tests first before executing any deployment steps.
- + Keep your pipeline execution times short so your team gets fast feedback.
- + Write simple, modular pipeline steps so they are easy to debug when they fail.

Watch Out For

- ! Hardcoding passwords or cloud credentials directly into your pipeline YAML files.
- ! Writing massive shell scripts inside your YAML instead of saving them in a separate script file.
- ! Forgetting to cache dependencies, which makes your pipelines run 10 times slower.
- ! Ignoring failing pipeline stages and merging broken code into your main branch anyway.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh